

주제 선정

기상과 지하철 열차 혼잡도 간
상관관계 분석 및 예측

변수명	변수 설명	내용	단위
tm	시간	날짜 및 시각 (지하철 운행시간 동안만 제공)	-
line	호선	지하철 호선	-
station_number	역번호	역 번호	-
station_name	역명	역명	-
direction	상하구분	지하철 상행 하행 구분	-
stn	AWS 지점 코드	AWS 지점 코드	-
ta	기온	정시 기온	°C
wd	풍향	정시 10분 평균 풍향	degree
ws	풍속	정시 10분 평균 풍속	m/s
m_day	일강수량	0 ~ 24시까지의 일 강수량	mm
m_hr1	시간 강수량	1시간 강수량	mm
hm	상대습도	정시 상대 습도	%
si	일사량	ASOS 일사량	MJ/m²
ta_chi	체감온도	500m 객관분석 자료	°C
congestion	혼잡도	열차 내 혼잡도	%

X

역번호
상·하행
기온
풍속
시간강수량 상
대습도 일사량
체감온도 (Class)

Y

특정 지하철역 혼잡도

현재 시간을 포함한 총 2시간 데이터를 이용,
1시간 뒤의 특정 지하철 역의 혼잡도 예측

원본 데이터

날씨마루 분석 플랫폼 제공 데이터

train21

```
origin_2021 = pd.read_csv('./train_subway21.csv', index_col=0)
origin_2021
```

	train_subway21.tm	train_subway21.line	train_subway21.station_number
1	2021010100	1	150
2	2021010101	1	150
3	2021010105	1	150
4	2021010106	1	150
5	2021010107	1	150
...	...	...	...
5452139	2021123119	8	2828
5452140	2021123120	8	2828
5452141	2021123121	8	2828
5452142	2021123122	8	2828
5452143	2021123123	8	2828

5452143 rows x 15 columns

train22

```
origin_2022 = pd.read_csv('./train_subway22.csv', index_col=0)
origin_2022
```

	train_subway22.tm	train_subway22.line	train_subway22.station_number
1	2022010100	1	150
2	2022010101	1	150
3	2022010105	1	150
4	2022010106	1	150
5	2022010107	1	150
...	...	...	...
5396196	2022123119	8	2828
5396197	2022123120	8	2828
5396198	2022123121	8	2828
5396199	2022123122	8	2828
5396200	2022123123	8	2828

5396200 rows x 15 columns

train23

```
origin_2023 = pd.read_csv('./train_subway23.csv', index_col=0)
origin_2023
```

	train_subway23.tm	train_subway23.line	train_subway23.station_number
1	2023010100	1	150
2	2023010101	1	150
3	2023010105	1	150
4	2023010106	1	150
5	2023010107	1	150
...	...	...	...
5520977	2023123119	8	2828
5520978	2023123120	8	2828
5520979	2023123121	8	2828
5520980	2023123122	8	2828
5520981	2023123123	8	2828

5520981 rows x 15 columns

칼럼명 변경 및 원본 데이터 통합

```
origin_2021 = origin_2021.rename(columns={
    'train_subway21.tm': 'time',
    'train_subway21.line': 'line',
    'train_subway21.station_number': 'station_number',
    'train_subway21.station_name': 'station_name',
    'train_subway21.direction': 'direction',
    'train_subway21.stn': 'AWS_num',
    'train_subway21.ta': 'temp',
    'train_subway21.wd': 'wind_direction',
    'train_subway21.ws': 'wind_speed',
    'train_subway21.rn_day': 'rn_day',
    'train_subway21.rn_hr1': 'rn_hr',
    'train_subway21.hm': 'humidity',
    'train_subway21.si': 'solar',
    'train_subway21.ta_chi': 'feel_temp',
    'train_subway21.congestion': 'congestion'
})
```

```
origin_2022 = origin_2022.rename(columns={
    'train_subway22.tm': 'time',
    'train_subway22.line': 'line',
    'train_subway22.station_number': 'station_number',
    'train_subway22.station_name': 'station_name',
    'train_subway22.direction': 'direction',
    'train_subway22.stn': 'AWS_num',
    'train_subway22.ta': 'temp',
    'train_subway22.wd': 'wind_direction',
    'train_subway22.ws': 'wind_speed',
    'train_subway22.rn_day': 'rn_day',
    'train_subway22.rn_hr1': 'rn_hr',
    'train_subway22.hm': 'humidity',
    'train_subway22.si': 'solar',
    'train_subway22.ta_chi': 'feel_temp',
    'train_subway22.congestion': 'congestion'
})
```

```
origin_2023 = origin_2023.rename(columns={
    'train_subway23.tm': 'time',
    'train_subway23.line': 'line',
    'train_subway23.station_number': 'station_number',
    'train_subway23.station_name': 'station_name',
    'train_subway23.direction': 'direction',
    'train_subway23.stn': 'AWS_num',
    'train_subway23.ta': 'temp',
    'train_subway23.wd': 'wind_direction',
    'train_subway23.ws': 'wind_speed',
    'train_subway23.rn_day': 'rn_day',
    'train_subway23.rn_hr1': 'rn_hr',
    'train_subway23.hm': 'humidity',
    'train_subway23.si': 'solar',
    'train_subway23.ta_chi': 'feel_temp',
    'train_subway23.congestion': 'congestion'
})
```

```
# origin dataframe 통합
origin_all = pd.concat([origin_2021, origin_2022, origin_2023], ignore_index=True)
origin_all
```

	time	line	station_number	station_name	direction	AWS_num	temp	wind_direction	wind_speed	rn_day	rn_hr	humidity	solar	feel_temp
0	2021010100	1	150	서울역	상선	419	-9.6	291.1	3.3	0.0	0.0	-99.0	-99.0	-12.6
1	2021010101	1	150	서울역	상선	419	-9.7	284.6	2.0	0.0	0.0	-99.0	-99.0	-9.8
2	2021010105	1	150	서울역	상선	419	-9.3	124.7	2.4	0.0	0.0	-99.0	-99.0	-10.3
3	2021010106	1	150	서울역	상선	419	-9.3	126.2	1.7	0.0	0.0	-99.0	-99.0	-10.1
4	2021010107	1	150	서울역	상선	419	-9.1	145.7	1.3	0.0	0.0	-99.0	-99.0	-9.7
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
16369319	2023123119	8	2828	남위례	하선	572	0.6	0.0	0.0	7.0	0.0	83.1	-99.0	0.0
16369320	2023123120	8	2828	남위례	하선	572	0.0	354.7	0.0	7.0	0.0	84.7	-99.0	-0.6
16369321	2023123121	8	2828	남위례	하선	572	-0.6	0.0	0.0	7.0	0.0	85.1	-99.0	-1.1
16369322	2023123122	8	2828	남위례	하선	572	-0.8	0.0	0.0	7.0	0.0	85.6	-99.0	-1.3
16369323	2023123123	8	2828	남위례	하선	572	-1.3	110.7	0.1	7.0	0.0	85.6	-99.0	-1.9

16369324 rows x 15 columns

LSTM (1)

: 데이터 전처리 - 결측치 처리

```
# 결측값 -> NaN
sentinels = [-99, -99.9, -999.0]
origin_all = origin_all.replace(sentinels, np.nan)

# 결측치 확인
print('결측치 처리 전 결측치 개수')
print(origin_all.isnull().sum())

결측치 처리 전 결측치 개수
time                0
line                0
station_number      0
station_name        0
direction            0
AWS_num             0
temp               216468
wind_direction      230786
wind_speed          230786
rn_day              351574
rn_hr               360796
humidity            844594
solar               6064242
feel_temp           352
congestion          0
dtype: int64
```

```
# 결측치 발생 원인 그룹화
target_cols = ['temp', 'wind_direction', 'wind_speed', 'rn_day', 'rn_hr', 'humidity', 'solar', 'feel_temp']

# AWS_num과 시간 기준 정렬
origin_all = origin_all.sort_values(by=['AWS_num', 'time'])

# 그룹별 보간
origin_all[target_cols] = (
    origin_all.groupby('AWS_num')[target_cols]
    .transform(lambda grp: grp.bfill().ffill())
)

# 시간 기준 재정렬
origin_all = origin_all.sort_values(by='time').reset_index(drop=True)
```

```
# 결측치 재확인
print('결측치 처리 후 결측치 개수')
print(origin_all.isnull().sum())

결측치 처리 후 결측치 개수
time                0
line                0
station_number      0
station_name        0
direction            0
AWS_num             0
temp               0
wind_direction      0
wind_speed          0
rn_day              0
rn_hr               0
humidity            0
solar               0
feel_temp           0
congestion          0
dtype: int64
```

Step 1: AWS_num → 시간 순으로 정렬
Step 2: bfill, ffill 수행
Step 3: 시간 기준 재정렬

LSTM (1)

: 데이터 전처리 - 파생 변수 및 추가 칼럼 생성 / 필요 칼럼 추출

```
# 상/하행 레이블 정의
direction_map = {'상선': 0, '하선': 1, '내선': 2, '외선': 3}

# 체감온도 구간 레이블 정의
bins = [-float('inf'), -10, 5, 20, 28, float('inf')]
labels = ["0", "1", "2", "3", "4"]

# 원시 DataFrame temp_df 정의
temp_df = origin_all.copy()

# direction을 숫자 레이블로 변환
temp_df['direction'] = temp_df['direction'].map(direction_map)

# 체감온도 단계 파생변수 추가
temp_df['feel_stage'] = pd.cut(temp_df['feel_temp'], bins=bins, labels=labels).astype(int)

# 필요한 칼럼만 추출
selected_cols = [
    'time',
    'station_number',
    'direction',
    'temp',
    'wind_speed',
    'rn_hr',
    'humidity',
    'solar',
    'congestion',
    'feel_stage'
]

# temp_df에서 필요한 칼럼만 train_data DataFrame에 저장
train_data = temp_df[selected_cols].copy()

# station_number, direction, time 순으로 정렬
train_data = train_data.sort_values(
    by=['station_number', 'direction', 'time']
).reset_index(drop=True)

# time 제거
train_data = train_data.iloc[:, 1:]
```

	station_number	direction	temp	wind_speed	rn_hr	humidity	solar	congestion	feel_stage
0	150	0	-9.6	3.3	0.0	47.5	0.0	0	0
1	150	0	-9.7	2.0	0.0	47.5	0.0	0	1
2	150	0	-9.3	2.4	0.0	47.5	0.0	1	0
3	150	0	-9.3	1.7	0.0	47.5	0.0	2	0
4	150	0	-9.1	1.3	0.0	47.5	0.0	3	1
...	...	...	...	...	...	...	...	...	...
16369319	9006	1	1.4	0.0	0.0	92.1	0.0	15	1
16369320	9006	1	0.8	0.5	0.0	95.3	0.0	13	1
16369321	9006	1	0.4	0.1	0.0	96.2	0.0	14	1
16369322	9006	1	0.0	0.1	0.0	97.1	0.0	14	1
16369323	9006	1	-0.2	0.2	0.0	97.6	0.0	12	1

16369324 rows x 9 columns

direction (열차 방향)

0: 상선
1: 하선
2: 내선
3: 외선

feel_stage (체감 온도)

0 : $x \leq -10$
1 : $-10 < x \leq 5$
2 : $5 < x \leq 20$
3 : $20 < x \leq 28$
4 : $28 < x$

LSTM (1)

: 슬라이딩 윈도우

```
def extract_inputoutput_shift(df, lookback_time=2, predict_time=1):  
    # 1) 과거 시점별 피쳐 생성  
    feat_dfs = []  
    for t in range(lookback_time):  
        tmp = df.shift(t)  
        feat_dfs.append(tmp)  
    X = pd.concat(feat_dfs, axis=1)  
  
    # 2) 예측 대상(타겟) 생성  
    y = df[['congestion']].shift(-predict_time)  
  
    # 3) NaN 제거 및 인덱스 리셋  
    valid_idx = X.dropna().index.intersection(y.dropna().index)  
    X = X.loc[valid_idx].reset_index(drop=True)  
    y = y.loc[valid_idx].reset_index(drop=True)  
  
    print("X, Y 데이터 분류 완료!")  
    return X, y
```

```
x, t = extract_inputoutput_shift(train_data)  
x
```

X, Y 데이터 분류 완료!

	station_number	direction	temp	wind_speed
0	150	0	-9.7	2.0
1	150	0	-9.3	2.4
2	150	0	-9.3	1.7
3	150	0	-9.1	1.3
4	150	0	-8.5	0.6
...	...	...	...	...
16369317	9006	1	2.6	0.3
16369318	9006	1	1.4	0.0
16369319	9006	1	0.8	0.5
16369320	9006	1	0.4	0.1
16369321	9006	1	0.0	0.1

16369322 rows x 18 columns

t	congestion
0	1.0
1	2.0
2	3.0
3	3.0
4	5.0
...	...
16369317	15.0
16369318	13.0
16369319	14.0
16369320	14.0
16369321	12.0

16369322 rows x 1 columns

LSTM (1)

: 데이터셋 분할 및 reshape

```
x_train, x_test, t_train, t_test = train_test_split(x, t, test_size=0.2, shuffle=False)
```

```
print('x_train shape :', x_train.shape)
```

```
print('t_train shape :', t_train.shape)
```

```
print('x_test shape :', x_test.shape)
```

```
print('t_test shape :', t_test.shape)
```

```
x_train shape : (13095457, 18)
```

```
t_train shape : (13095457, 1)
```

```
x_test shape : (3273865, 18)
```

```
t_test shape : (3273865, 1)
```

```
timesteps = 2
```

```
feature = 9
```

```
x_train = np.array(x_train)
```

```
x_train = x_train.reshape(x_train.shape[0], timesteps, feature)
```

```
x_test = np.array(x_test)
```

```
x_test = x_test.reshape(x_test.shape[0], timesteps, feature)
```

```
t_train = np.array(t_train)
```

```
t_test = np.array(t_test)
```

```
print('reshape 후 x_train shape :', x_train.shape)
```

```
print('t_train shape :', t_train.shape)
```

```
print('reshape 후 x_test shape :', x_test.shape)
```

```
print('t_test shape :', t_test.shape)
```

```
reshape 후 x_train shape : (13095457, 2, 9)
```

```
t_train shape : (13095457, 1)
```

```
reshape 후 x_test shape : (3273865, 2, 9)
```

```
t_test shape : (3273865, 1)
```

LSTM (1)

: 모델 구성

```
cell_size = 128
timesteps = 2
feature = 9

model = Sequential(name='congestion_lstm')

model.add(LSTM(cell_size, input_shape=(timesteps, feature), return_sequences=True))
model.add(LSTM(cell_size))

model.add(Dense(1))

model.compile(loss=mse, optimizer=Adam(learning_rate = 0.001), metrics=[RootMeanSquaredError()])
model.summary()
```

```
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not
to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer
super().__init__(**kwargs)
```

Model: "congestion_lstm"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 2, 128)	70,656
lstm_1 (LSTM)	(None, 128)	131,584
dense (Dense)	(None, 1)	129

Total params: 202,369 (790.50 KB)

Trainable params: 202,369 (790.50 KB)

Non-trainable params: 0 (0.00 B)

cell size: 128

timesteps: 2

feature: 9

LSTM (1)

: 모델 학습

```
model.fit(x_train, t_train, epochs = 20, batch_size = 32)
```

```
Epoch 1/20  
409234/409234 ————— 835s 2ms/step - loss: 108.5356 - root_mean_squared_error: 10.4032  
Epoch 2/20  
409234/409234 ————— 870s 2ms/step - loss: 91.4314 - root_mean_squared_error: 9.5618  
Epoch 3/20  
409234/409234 ————— 3028s 7ms/step - loss: 87.6398 - root_mean_squared_error: 9.3615  
Epoch 4/20  
409234/409234 ————— 925s 2ms/step - loss: 84.3445 - root_mean_squared_error: 9.1839  
Epoch 5/20  
409234/409234 ————— 832s 2ms/step - loss: 82.5521 - root_mean_squared_error: 9.0858  
Epoch 6/20  
409234/409234 ————— 934s 2ms/step - loss: 80.5084 - root_mean_squared_error: 8.9726  
Epoch 7/20  
409234/409234 ————— 933s 2ms/step - loss: 78.9541 - root_mean_squared_error: 8.8856  
Epoch 8/20  
409234/409234 ————— 909s 2ms/step - loss: 79.2485 - root_mean_squared_error: 8.9021  
Epoch 9/20  
409234/409234 ————— 937s 2ms/step - loss: 77.2708 - root_mean_squared_error: 8.7904  
Epoch 10/20  
409234/409234 ————— 976s 2ms/step - loss: 76.7233 - root_mean_squared_error: 8.7592  
Epoch 11/20  
409234/409234 ————— 967s 2ms/step - loss: 76.1620 - root_mean_squared_error: 8.7271  
Epoch 12/20  
409234/409234 ————— 970s 2ms/step - loss: 76.2445 - root_mean_squared_error: 8.7317  
Epoch 13/20  
409234/409234 ————— 981s 2ms/step - loss: 75.3809 - root_mean_squared_error: 8.6822  
Epoch 14/20  
409234/409234 ————— 944s 2ms/step - loss: 75.9575 - root_mean_squared_error: 8.7153  
Epoch 15/20  
409234/409234 ————— 962s 2ms/step - loss: 75.7193 - root_mean_squared_error: 8.7017  
Epoch 16/20  
409234/409234 ————— 994s 2ms/step - loss: 75.6455 - root_mean_squared_error: 8.6974  
Epoch 17/20  
409234/409234 ————— 976s 2ms/step - loss: 75.2983 - root_mean_squared_error: 8.6773  
Epoch 18/20  
409234/409234 ————— 1006s 2ms/step - loss: 77.1958 - root_mean_squared_error: 8.7861  
Epoch 19/20  
409234/409234 ————— 977s 2ms/step - loss: 75.7195 - root_mean_squared_error: 8.7017  
Epoch 20/20  
409234/409234 ————— 1026s 3ms/step - loss: 75.3083 - root_mean_squared_error: 8.6780
```

epochs: 20

batch size: 32

train RMSE: 8.6780

LSTM (1)

: 모델 검증

```
pred = model.predict(x_test)

for i in range(1, 10):
    print('혼잡도 예측: ', round(pred[i][0]), '정답: ', round(t_test[i][0], 2))
```

102309/102309 ————— 64s 625us/step

혼잡도 예측: 3 정답: 9.0
혼잡도 예측: 9 정답: 13.0
혼잡도 예측: 17 정답: 18.0
혼잡도 예측: 16 정답: 13.0
혼잡도 예측: 15 정답: 9.0
혼잡도 예측: 5 정답: 12.0
혼잡도 예측: 18 정답: 5.0
혼잡도 예측: 5 정답: 3.0
혼잡도 예측: 6 정답: 2.0

```
loss, rmse = model.evaluate(x_test, t_test, verbose=1)
print('test loss (MSE): ', round(loss, 6))
print('test RMSE: ', round(rmse, 2))
```

102309/102309 ————— 62s 601us/step - loss: 145.2646 - root_mean_squared_error: 11.8708
test loss (MSE): 156.932266
test RMSE: 12.53

test RMSE: 12.53

LSTM (2)

: 데이터 전처리 - 결측치 처리

```
target_cols = ['temp', 'wind_direction', 'wind_speed', 'rn_day', 'rn_hr', 'humidity', 'solar', 'feel_temp']
specific_cols = ['wind_direction', 'rn_day', 'rn_hr']
linear_cols = ['temp', 'wind_speed', 'humidity', 'solar', 'feel_temp']

# 1단계: 관측소별 ffill, bfill로 보간방식
print('1단계시 결측치 값 변화')
for col in specific_cols:
    df[col] = df.groupby('AWS_num')[col].ffill().bfill()
print(df.isnull().sum())
print('-----')

# 2단계: 관측소별 선형보간 방식
print('2단계시 결측치 값 변화')
df[linear_cols] = df.groupby('AWS_num')[linear_cols].transform(
    lambda x: x.interpolate(method='linear')
)
print(df.isnull().sum())
print('-----')

# 3단계: 경계값 처리
print('3단계시 결측치 값 변화')
for col in linear_cols:
    df[col] = df.groupby('AWS_num')[col].ffill().bfill()
print(df.isnull().sum())
print('-----')
```

1단계시 결측치 값 변화

time	0
line	0
station_number	0
station_name	0
direction	0
AWS_num	0
temp	216468
wind_direction	0
wind_speed	230786
rn_day	0
rn_hr	0
humidity	844594
solar	6064242
feel_temp	352
congestion	0
dtype: int64	

2단계시 결측치 값 변화

time	0
line	0
station_number	0
station_name	0
direction	0
AWS_num	0
temp	4634
wind_direction	0
wind_speed	4636
rn_day	0
rn_hr	0
humidity	633392
solar	3440
feel_temp	0
congestion	0
dtype: int64	

3단계시 결측치 값 변화

time	0
line	0
station_number	0
station_name	0
direction	0
AWS_num	0
temp	0
wind_direction	0
wind_speed	0
rn_day	0
rn_hr	0
humidity	0
solar	0
feel_temp	0
congestion	0
dtype: int64	

Step 1: 관측소 별 ffill, bfill을 이용한 보간

Step 2: 관측소 별 선형보간

Step 3: 경계값 처리

LSTM (2)

: 데이터 전처리 - 파생 변수 및 추가 칼럼 생성

```
# 상/하행 레이블 정의
direction_map = {'상선': 0, '하선': 1, '내선': 2, '외선': 3}

# 체감온도 구간 레이블 정의
bins = [-float('inf'), -10, 5, 20, 28, float('inf')]
labels = ["0", "1", "2", "3", "4"]

# 임시 DataFrame temp_df 생성
temp_df = df.copy()

# direction을 숫자 레이블로 변환
temp_df['direction'] = temp_df['direction'].map(direction_map)

# 체감온도 단계 파생변수 추가
temp_df['feel_stage'] = pd.cut(temp_df['feel_temp'], bins=bins, labels=labels).astype(int)

holidays = [
    # 2021
    '20210101', '20210211', '20210212', '20210213', '20210301',
    '20210505', '20210519', '20210606', '20210815', '20210920',
    '20210921', '20210922', '20211003', '20211009', '20211225',
    # 2022
    '20220101', '20220131', '20220201', '20220202', '20220301',
    '20220309', '20220505', '20220508', '20220601', '20220606',
    '20220815', '20220909', '20220910', '20220911', '20220912',
    '20221003', '20221009', '20221010', '20221225',
    # 2023
    '20230101', '20230121', '20230122', '20230123', '20230124',
    '20230301', '20230505', '20230527', '20230529', '20230606',
    '20230815', '20230928', '20230929', '20230930', '20231002',
    '20231003', '20231009', '20231225'
]

# 공휴일 리스트를 set으로 변환하여 조회 속도 향상
holidays_set = set(holidays)

# 'time' 컬럼을 datetime 형태로 변환
temp_df['time'] = pd.to_datetime(temp_df['time'], format='%Y%m%d%H')

# 'is_holiday' 컬럼 초기화 (기본값 0: 평일)
temp_df['is_holiday'] = 0

# 주말(토/일) 확인하여 1로 설정
is_weekend = {temp_df['time'].dt.dayofweek >= 5}
temp_df.loc[is_weekend, 'is_holiday'] = 1

# 공휴일 확인하여 1로 설정
is_in_holidays = temp_df['time'].dt.strftime('%Y%m%d').isin(holidays_set)
temp_df.loc[is_in_holidays, 'is_holiday'] = 1
```

direction (열차 방향)

- 0: 상선
- 1: 하선
- 2: 내선
- 3: 외선

feel_stage (체감 온도)

- 0 : $x \leq -10$
- 1 : $-10 < x \leq 5$
- 2 : $5 < x \leq 20$
- 3 : $20 < x \leq 28$
- 4 : $28 < x$

Is_holiday (휴일 여부)

- 0: 평일
- 1: 주말 또는 공휴일

LSTM (2)

: 데이터 전처리 - 필요 칼럼 추출

```
# 필요한 컬럼만 추출
selected_cols = [
    'time',
    'station_number',
    'direction',
    'temp',
    'wind_speed',
    'rn_hr',
    'humidity',
    'solar',
    'congestion',
    'feel_stage',
    'is_holiday'
]

# temp_df에서 필요한 컬럼만 train_data DataFrame에 저장
train_data = temp_df[selected_cols].copy()

# station_number, direction, time 순으로 정렬
train_data = train_data.sort_values(
    by=['station_number', 'direction', 'time']
).reset_index(drop=True)

# time 제거
train_data = train_data.iloc[:, 1:]

# 완성
train_data
```

	station_number	direction	temp	wind_speed	rn_hr	humidity	solar	congestion	feel_stage	is_holiday
0	150	0	-9.6	3.3	0.0	47.5	0.00	0	0	1
1	150	0	-9.7	2.0	0.0	47.5	0.00	0	1	1
2	150	0	-9.3	2.4	0.0	47.5	0.00	1	0	1
3	150	0	-9.3	1.7	0.0	47.5	0.00	2	0	1
4	150	0	-9.1	1.3	0.0	47.5	0.00	3	1	1
...	...	...	...	...	...	...	...	...	...	...
16369319	9006	1	1.4	0.0	0.5	92.1	0.03	15	1	1
16369320	9006	1	0.8	0.5	0.5	95.3	0.03	13	1	1
16369321	9006	1	0.4	0.1	0.5	96.2	0.03	14	1	1
16369322	9006	1	0.0	0.1	0.5	97.1	0.03	14	1	1
16369323	9006	1	-0.2	0.2	0.5	97.6	0.03	12	1	1
16369324 rows x 10 columns										

LSTM (2)

: 슬라이딩 윈도우

```
def extract_inputoutput_shift(df, lookback_time=2, predict_time=1):  
    # 1) 과거 시점별 피쳐 생성  
    feat_dfs = []  
    for t in range(lookback_time):  
        tmp = df.shift(t)  
        feat_dfs.append(tmp)  
    X = pd.concat(feat_dfs, axis=1)  
  
    # 2) 예측 대상(타겟) 생성  
    y = df[['congestion']].shift(-predict_time)  
  
    # 3) NaN 제거 및 인덱스 리셋  
    valid_idx = X.dropna().index.intersection(y.dropna().index)  
    X = X.loc[valid_idx].reset_index(drop=True)  
    y = y.loc[valid_idx].reset_index(drop=True)  
  
    print("X, Y 데이터 분류 완료!")  
    return X, y
```

```
x, t = extract_inputoutput_shift(train_data)  
x
```

X, Y 데이터 분류 완료!

	station_number	direction	temp	wind_speed
0	150	0	-9.7	2.0
1	150	0	-9.3	2.4
2	150	0	-9.3	1.7
3	150	0	-9.1	1.3
4	150	0	-8.5	0.6
...	...	...	...	...
16369317	9006	1	2.6	0.3
16369318	9006	1	1.4	0.0
16369319	9006	1	0.8	0.5
16369320	9006	1	0.4	0.1
16369321	9006	1	0.0	0.1

16369322 rows x 18 columns

t

congestion

0	1.0
1	2.0
2	3.0
3	3.0
4	5.0
...	...
16369317	15.0
16369318	13.0
16369319	14.0
16369320	14.0
16369321	12.0

16369322 rows x 1 columns

LSTM (2)

: 데이터셋 분할 및 reshape

```
x_train, x_test, t_train, t_test = train_test_split(x, t, test_size=0.2, shuffle=False)

print('x_train shape :', x_train.shape)
print('t_train shape :', t_train.shape)
print('x_test shape :', x_test.shape)
print('t_test shape :', t_test.shape)

x_train shape : (13095457, 20)
t_train shape : (13095457, 1)
x_test shape : (3273865, 20)
t_test shape : (3273865, 1)

timesteps = 2
feature = 10

x_train = np.array(x_train)
x_train = x_train.reshape(x_train.shape[0], timesteps, feature)

x_test = np.array(x_test)
x_test = x_test.reshape(x_test.shape[0], timesteps, feature)

t_train = np.array(t_train)
t_test = np.array(t_test)

print('reshape 후 x_train shape :', x_train.shape)
print('t_train shape :', t_train.shape)
print('reshape 후 x_test shape :', x_test.shape)
print('t_test shape :', t_test.shape)

reshape 후 x_train shape : (13095457, 2, 10)
t_train shape : (13095457, 1)
reshape 후 x_test shape : (3273865, 2, 10)
t_test shape : (3273865, 1)
```

LSTM (2)

: 모델 구성

```
def lstm_model(cell_size=128, learning_rate=0.001):  
    timesteps = 2  
    feature = 10  
    model = Sequential(name='congestion_lstm_2')  
    model.add(LSTM(cell_size, input_shape=(timesteps, feature), return_sequences=True,  
                  kernel_regularizer=regularizers.l2(0.001)))  
    model.add(LSTM(cell_size, kernel_regularizer=regularizers.l2(0.001)))  
    model.add(Dropout(0.2))  
    model.add(Dense(1))  
    model.compile(loss='mse', optimizer=Adam(learning_rate=learning_rate), metrics=[RootMeanSquaredError()])  
    return model
```

```
temp_lstm = lstm_model()  
temp_lstm.summary()
```

```
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shape`  
to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model.  
super().__init__(**kwargs)
```

Model: "congestion_lstm_2"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 2, 128)	71,168
lstm_1 (LSTM)	(None, 128)	131,584
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 1)	129

Total params: 202,881 (792.50 KB)

Trainable params: 202,881 (792.50 KB)

Non-trainable params: 0 (0.00 B)

- LSTM 2층
- Dropout 1층 (과적합 억제)
- Dense 1층 (출력)

LSTM (2)

: 하이퍼파라미터 튜닝

```
early_stopping = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)

# 성능 평가 함수
def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))
rmse_scorer = make_scorer(rmse, greater_is_better=False)

# 데이터가 너무 많아 약 20%만 추출하여 하이퍼파라미터를 탐색하도록
x_train_subset, _, t_train_subset, _ = train_test_split(
    x_train, t_train, train_size=0.2, shuffle=False, random_state=42
)

def objective(trial):
    cell_size = trial.suggest_categorical('cell_size', [64, 128])
    learning_rate = trial.suggest_float('learning_rate', 5e-4, 1e-3, log=True)
    epochs = trial.suggest_int('epochs', 10, 20)
    batch_size = trial.suggest_categorical('batch_size', [256, 512])

    # scikeras로 래핑
    Congestion_LSTM_MODEL = KerasRegressor(
        model=lstm_model,
        cell_size=cell_size,
        learning_rate=learning_rate,
        epochs=epochs,
        batch_size=batch_size,
        callbacks=[early_stopping]
    )

    # cross_val_score에서 cv=2와 데이터 직접 전달
    scores = cross_val_score(Congestion_LSTM_MODEL, x_train_subset, t_train_subset, scoring=rmse_scorer, cv=2)
    return -scores.mean()

bayesian_study = optuna.create_study(direction='minimize')

bayesian_study.optimize(objective, n_trials=20)

print("Random Search에서의 최고의 하이퍼파라미터 조합 :", bayesian_study.best_params)
```

Random Search에서의 최고의 하이퍼파라미터 조합 : {'cell_size': 128, 'learning_rate': 0.0007746521536265282, 'epochs': 20, 'batch_size': 256}

Optuna를 활용한 하이퍼파라미터 튜닝

- objective() 함수 20번 실행
- trial마다 epochs 값이 10~20 사이에서 샘플링
 - 최대 400 epochs (early stop으로 단축 가능)
- 각 trial에서 2-fold 교차 검증 수행 (2회 반복)

→ 최적의 하이퍼파라미터 조합 추출

LSTM (2)

: 모델 학습

```
best_params = bayesian_study.best_params
final_lstm_model = lstm_model(
    cell_size=best_params['cell_size'],
    learning_rate=best_params['learning_rate']
)

history = final_lstm_model.fit(
    x_train, t_train,
    epochs=best_params['epochs'],
    batch_size=best_params['batch_size'],
    callbacks=[early_stopping]
)
```

- 최적의 cell_size, learning_rate를 이용한 LSTM 모델 구성
- early_stopping을 통한 overfitting 방지

train RMSE: 8.02

```
Epoch 1/20
51155/51155 ————— 321s 6ms/step - loss: 106.2686 - root_mean_squared_error: 10.2081
Epoch 2/20
/opt/anaconda3/lib/python3.12/site-packages/keras/src/callbacks/early_stopping.py:153: UserWarning
Early stopping conditioned on metric 'val_loss' which is not available. Available metrics are: loss,
root_mean_squared_error
    current = self.get_monitor_value(logs)
51155/51155 ————— 301s 6ms/step - loss: 85.5827 - root_mean_squared_error: 9.1661
Epoch 3/20
51155/51155 ————— 320s 6ms/step - loss: 80.2262 - root_mean_squared_error: 8.8443
Epoch 4/20
51155/51155 ————— 318s 6ms/step - loss: 76.9557 - root_mean_squared_error: 8.6370
Epoch 5/20
51155/51155 ————— 316s 6ms/step - loss: 75.2132 - root_mean_squared_error: 8.5240
Epoch 6/20
51155/51155 ————— 320s 6ms/step - loss: 73.7317 - root_mean_squared_error: 8.4263
Epoch 7/20
51155/51155 ————— 317s 6ms/step - loss: 73.8392 - root_mean_squared_error: 8.4299
Epoch 8/20
51155/51155 ————— 304s 6ms/step - loss: 73.1968 - root_mean_squared_error: 8.3876
Epoch 9/20
51155/51155 ————— 313s 6ms/step - loss: 72.1967 - root_mean_squared_error: 8.3238
Epoch 10/20
51155/51155 ————— 322s 6ms/step - loss: 72.1289 - root_mean_squared_error: 8.3157
Epoch 11/20
51155/51155 ————— 335s 7ms/step - loss: 72.1551 - root_mean_squared_error: 8.3144
Epoch 12/20
51155/51155 ————— 334s 7ms/step - loss: 71.3339 - root_mean_squared_error: 8.2622
Epoch 13/20
51155/51155 ————— 323s 6ms/step - loss: 71.0870 - root_mean_squared_error: 8.2450
Epoch 14/20
51155/51155 ————— 317s 6ms/step - loss: 70.7383 - root_mean_squared_error: 8.2209
Epoch 15/20
51155/51155 ————— 317s 6ms/step - loss: 69.6614 - root_mean_squared_error: 8.1523
Epoch 16/20
51155/51155 ————— 320s 6ms/step - loss: 69.2758 - root_mean_squared_error: 8.1262
Epoch 17/20
51155/51155 ————— 317s 6ms/step - loss: 69.2292 - root_mean_squared_error: 8.1185
Epoch 18/20
51155/51155 ————— 323s 6ms/step - loss: 68.6245 - root_mean_squared_error: 8.0799
Epoch 19/20
51155/51155 ————— 329s 6ms/step - loss: 68.2942 - root_mean_squared_error: 8.0568
Epoch 20/20
51155/51155 ————— 316s 6ms/step - loss: 67.7238 - root_mean_squared_error: 8.0195
```

LSTM (2)

: 모델 검증

```
pred = final_lstm_model.predict(x_test)

for i in range(0, 20):
    print('혼잡도 예측 : ', round(pred[i][0], 2), '/정답 : ', round(t_test[i][0], 2))
```

102309/102309 ————— 59s 571us/step

혼잡도 예측 : 4.66 /정답 : 8.0
혼잡도 예측 : 8.96 /정답 : 9.0
혼잡도 예측 : 9.35 /정답 : 13.0
혼잡도 예측 : 12.87 /정답 : 18.0
혼잡도 예측 : 19.67 /정답 : 13.0
혼잡도 예측 : 9.77 /정답 : 9.0
혼잡도 예측 : 7.06 /정답 : 12.0
혼잡도 예측 : 13.43 /정답 : 5.0
혼잡도 예측 : 2.3 /정답 : 3.0
혼잡도 예측 : 3.62 /정답 : 2.0
혼잡도 예측 : 3.21 /정답 : 0.0
혼잡도 예측 : 2.44 /정답 : 6.0
혼잡도 예측 : 4.21 /정답 : 5.0
혼잡도 예측 : 3.78 /정답 : 7.0
혼잡도 예측 : 8.83 /정답 : 5.0
혼잡도 예측 : 3.3 /정답 : 3.0
혼잡도 예측 : 4.97 /정답 : 4.0
혼잡도 예측 : 1.98 /정답 : 6.0
혼잡도 예측 : 3.99 /정답 : 6.0
혼잡도 예측 : 3.96 /정답 : 6.0

```
loss, rmse = final_lstm_model.evaluate(x_test, t_test, verbose=1)
print('test loss(MSE) : ', round(loss, 6))
print('test RMSE : ', round(rmse, 2))
```

102309/102309 ————— 63s 619us/step - loss: 144.2846 - root_mean_squared_error: 11.6320
test loss(MSE) : 136.410263
test RMSE : 11.53

test RMSE: 11.53

GRU

: 데이터 전처리 - 이상치 처리

```
#이상치
def find_iqr_outliers(series):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return (series < lower) | (series > upper)

num_cols = ['temp', 'wind_direction', 'wind_speed', 'rn_day', 'rn_hr', 'humidity', 'solar', 'feel_temp']

for col in num_cols:
    outlier_mask = find_iqr_outliers(origin_all[col])
    origin_all.loc[outlier_mask, col] = np.nan # 이상치만 NaN 처리
```

IQR 방식

데이터의 중앙 50%를 나타내는 사분위수 범위(IQR)를
기준으로 이상치를 정의하는 방법

→ 이상치들은 NaN으로 치환

GRU

: 데이터 전처리 - 결측치 처리 (1)

```
df['time'] = pd.to_datetime(df['time'], format='%Y%m%d%H')
df['month'] = df['time'].dt.month
df['hour'] = df['time'].dt.hour

# 2. 겨울/여름 마스크 (벡터화)
winter_mask = df['month'].isin([12, 1, 2])
summer_mask = df['month'].isin([6, 7, 8])
remain = df['month'].isin([3,4,5, 9, 10, 11])

# 3. 밤시간 (벡터화)
df['is_night'] = (
    (winter_mask & ((df['hour'] >= 20) | (df['hour'] < 5))) |
    (summer_mask & ((df['hour'] >= 22) | (df['hour'] < 5))) |
    (remain & ((df['hour'] >= 21) | (df['hour'] < 5)))
)
df.loc[df['is_night'] & df['solar'].isnull(), 'solar'] = int(0)

# 임시 컬럼 제거
df.drop(['month', 'hour', 'is_night'], axis=1, inplace=True)
```

결측치가 가장 많은 일사량의 결측치 처리를
최우선 수행

여름, 겨울, 봄+가을 3가지로 구분
일몰과 일출을 기준으로 결측치가 있다면 0으로
변환

GRU

: 데이터 전처리 - 결측치 처리 (2)

```
target_cols = ['temp', 'wind_direction', 'wind_speed', 'rn_day', 'rn_hr', 'humidity', 'solar', 'feel_temp']
specific_cols = ['wind_direction', 'rn_day', 'rn_hr']
linear_cols = ['temp', 'wind_speed', 'humidity', 'solar', 'feel_temp']
# 0단계: 동일 관측소그룹별 specific_cols를 시간순으로 bfill/ffill
print('0단계 시작')
for col in specific_cols:
    df[col] = (
        df.groupby('AWS_num')[col]
        .apply(lambda x: x.bfill().ffill()) # 각 그룹에서만 보간
        .reset_index(drop=True)
    )
print(df.isnull().sum())
print('-----')

# 1단계 : 동일 관측소그룹별 linear_cols를 시간순으로 선형보간
print('1단계 시작')
for col in linear_cols:
    df[col] = (
        df
        .sort_values(['AWS_num', 'time']) # 그룹 내 정렬!
        .groupby('AWS_num')[col]
        .apply(lambda x: x.interpolate(method='linear')) # 각 그룹에서만 보간
        .reset_index(drop=True)
    )
print(df.isnull().sum())
print('-----')

print('2단계 시작')
for col in linear_cols:
    # 1) 그룹 내 중앙값으로 채우기
    df[col] = (
        df.groupby('AWS_num')[col]
        .apply(lambda x: x.fillna(x.median()))
        .reset_index(drop=True)
    )
    # 2) 그래도 남으면 전체 중앙값
    df[col] = df[col].fillna(df[col].median())
print(df.isnull().sum())
print('-----')
```

0단계

동일 관측소(AWS)별 시간순으로 그룹안에서
풍향, 강수량(day), 강수량(hr)을 bfill, ffill로 보간

1단계

동일 관측소(AWS)별 시간순으로 그룹안에서만
기온, 풍속, 습도, 일사량, 체감온도를 선형보간

2단계

남은결측치들을 그룹내 중앙값으로 보간

GRU

: 데이터 전처리 - 결과

처리 전

결측치 처리 전	결측치 개수
time	0
line	0
station_number	0
station_name	0
direction	0
AWS_num	0
temp	216672
wind_direction	230786
wind_speed	714650
rn_day	3241523
rn_hr	1293576
humidity	844594
solar	6064242
feel_temp	1076
congestion	0
dtype:	int64

0단계

0단계 시작	
time	0
line	0
station_number	0
station_name	0
direction	0
AWS_num	0
temp	216672
wind_direction	0
wind_speed	714650
rn_day	0
rn_hr	0
humidity	844594
solar	2220357
feel_temp	1076
congestion	0
dtype:	int64

1단계

1단계 시작	
time	0
line	0
station_number	0
station_name	0
direction	0
AWS_num	0
temp	4634
wind_direction	0
wind_speed	4636
rn_day	0
rn_hr	0
humidity	633392
solar	0
feel_temp	0
congestion	0
dtype:	int64

최종

2단계 시작	
time	0
line	0
station_number	0
station_name	0
direction	0
AWS_num	0
temp	0
wind_direction	0
wind_speed	0
rn_day	0
rn_hr	0
humidity	0
solar	0
feel_temp	0
congestion	0
dtype:	int64

GRU

: 데이터 전처리 - 파생 변수 및 추가 칼럼 생성

```
# 상/하행 레이블 정의
direction_map = {'상선': 0, '하선': 1, '내선': 2, '외선': 3}

# 체감온도 구간 레이블 정의
bins = [-float('inf'), -10, 5, 20, 28, float('inf')]
labels = ["0", "1", "2", "3", "4"]

# 임시 DataFrame temp_df 정의
temp_df = df.copy()

# direction을 숫자 레이블로 변환
temp_df['direction'] = temp_df['direction'].map(direction_map)

# 체감온도 단계 파생변수 추가
temp_df['feel_stage'] = pd.cut(temp_df['feel_temp'], bins=bins, labels=labels).astype(int)
```

```
holidays = [
    # 2021
    '20210101', '20210211', '20210212', '20210213', '20210301',
    '20210505', '20210519', '20210606', '20210815', '20210920',
    '20210921', '20210922', '20211003', '20211009', '20211225',
    # 2022
    '20220101', '20220131', '20220201', '20220202', '20220301',
    '20220309', '20220505', '20220508', '20220601', '20220606',
    '20220815', '20220909', '20220910', '20220911', '20220912',
    '20221003', '20221009', '20221010', '20221225',
    # 2023
    '20230101', '20230121', '20230122', '20230123', '20230124',
    '20230301', '20230505', '20230527', '20230529', '20230606',
    '20230815', '20230928', '20230929', '20230930', '20231002',
    '20231003', '20231009', '20231225'
]

# 공휴일 리스트를 set으로 변환하여 조회 속도 향상
holidays_set = set(holidays)

# 2. 'is_holiday' 칼럼 초기화 (기본값 0: 평일)
temp_df['is_holiday'] = 0

# 3. 주말(토/일) 확인하여 1로 설정
# .dt.dayofweek는 요일을 숫자로 반환합니다 (월=0, 화=1, ..., 토=5, 일=6)
is_weekend = (temp_df['time'].dt.dayofweek >= 5)
temp_df.loc[is_weekend, 'is_holiday'] = 1

# 4. 공휴일 확인하여 1로 설정 (주말과 겹치더라도 1로 유지됨)
# .dt.strftime('%Y%m%d')로 날짜 부분만 YYYYMMDD 문자열로 변환하여 공휴일 리스트와 비교
is_in_holidays = temp_df['time'].dt.strftime('%Y%m%d').isin(holidays_set)
temp_df.loc[is_in_holidays, 'is_holiday'] = 1
```

direction (열차 방향)

- 0: 상선
- 1: 하선
- 2: 내선
- 3: 외선

feel_stage (체감 온도)

- 0 : $x \leq -10$
- 1 : $-10 < x \leq 5$
- 2 : $5 < x \leq 20$
- 3 : $20 < x \leq 28$
- 4 : $28 < x$

Is_holiday (휴일 여부)

- 0: 평일
- 1: 주말 또는 공휴일

GRU

: 데이터 전처리 - 필요 칼럼 추출

station_number / direction / temp / wind_speed / rn_hr / humidity / solar / congestion / feel_temp / feel_stage / is_holiday

	station_number	direction	temp	wind_speed	rn_hr	humidity	solar	congestion	feel_temp	feel_stage	is_holiday
0	150	0	-9.6	3.3	0.0	64.1	0.000000	0	-12.6	0	1
1	150	0	-9.7	2.0	0.0	64.1	0.000000	0	-9.8	1	1
2	150	0	-9.3	2.4	0.0	64.1	0.000000	1	-10.3	0	1
3	150	0	-9.3	1.7	0.0	64.1	0.000000	2	-10.1	0	1
4	150	0	-9.1	1.3	0.0	64.1	0.000000	3	-9.7	1	1
...	...	...	...	...	...	...	...	...	...	...	...
16369319	9006	1	1.4	0.0	0.0	92.1	0.007959	15	1.3	1	1
16369320	9006	1	0.8	0.5	0.0	95.3	0.000000	13	0.7	1	1
16369321	9006	1	0.4	0.1	0.0	96.2	0.000000	14	0.2	1	1
16369322	9006	1	0.0	0.1	0.0	97.1	0.000000	14	0.0	1	1
16369323	9006	1	-0.2	0.2	0.0	97.6	0.000000	12	-0.4	1	1

16369324 rows × 11 columns

GRU

: 모델 구성

```
from tensorflow.keras import regularizers
def gru_model(cell_size=128, learning_rate=0.001):
    timesteps = 2
    feature = 11
    model = Sequential(name='congestion_gru')
    model.add(GRU(cell_size, input_shape=(timesteps, feature), return_sequences=True,
                  kernel_regularizer=regularizers.l2(0.001)))
    model.add(GRU(cell_size, kernel_regularizer=regularizers.l2(0.001)))
    model.add(Dropout(0.2))
    model.add(Dense(1))
    model.compile(loss='mse', optimizer=Adam(learning_rate=learning_rate), metrics=[RootMeanSquaredError()])
    return model
```

- GRU 2층 (각 층마다 l2로 과적합억제)
- Dropout 1층 (과적합 억제)
- Dense 1층 (출력)

Model: "congestion_gru"

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 2, 128)	53,760
gru_1 (GRU)	(None, 128)	99,072
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 1)	129

Total params: 152,961 (597.50 KB)

Trainable params: 152,961 (597.50 KB)

Non-trainable params: 0 (0.00 B)

GRU

: 하이퍼파라미터 튜닝

```
early_stopping = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)

#성능 평가 함수(scoring)로 평균제곱근오차 기본 제공이 아니라 커스텀
def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))
rmse_scorer = make_scorer(rmse, greater_is_better=False)

#데이터가 너무 많아 10~20%만 추출하여 하이퍼파라미터 탐색하도록(더 빠르게)
x_train_subset, _, t_train_subset, _ = train_test_split(
    x_train, t_train, train_size=0.1, shuffle=False, random_state=42
)

def objective(trial):
    # Optuna에서 베이지안 방식으로 샘플링을 파라미터 정의
    cell_size = trial.suggest_categorical('cell_size', [128, 248])
    learning_rate = trial.suggest_float('learning_rate', 5e-4, 1e-3, log=True)
    epochs = trial.suggest_int('epochs', 15, 25)
    batch_size = trial.suggest_categorical('batch_size', [128, 256])

    # scikeras로 래핑
    Congestion_Gru_MODEL = KerasRegressor(
        model=gru_model,
        cell_size=cell_size,
        learning_rate=learning_rate,
        epochs=epochs,
        batch_size=batch_size,
        callbacks=[early_stopping]
    )
    # cross_val_score에서 cv=2와 데이터 직접 전달
    scores = cross_val_score(Congestion_Gru_MODEL, x_train_subset, t_train_subset, scoring=rmse_scorer, cv=2)
    return -scores.mean() # minimize(RMSE)
```

- Early stopping

조기 종료 및 고성능의 모델 가중치로 복원
과적합 방지, 학습시간 단축

- train data 서브셋 추출

튜닝/서치 속도 극대화

- Optuna

베이지안 방식으로 파라미터값 자동 샘플링

- scikeras.KerasRegressor

keras 모델을 sklearn으로 래핑

- sklearn cross-validation

검증셋 분리 및 성능 신뢰도 향상

파라미터 범위

cell_size: 128, 248

learning_rate: 5e-4, 1e-3

epochs: 15, 25

batch_size: 128, 256

결과

cell_size: 248

learning_rate: 0.00099

epochs: 18

batch_size: 128

GRU

: 모델 학습

```
Epoch 1/18
89520/89520 ————— 466s 5ms/step - loss: 111.5431 - root_mean_squared_error: 10.4858
Epoch 2/18
/usr/local/lib/python3.11/dist-packages/keras/src/callbacks/early_stopping.py:153: UserWarning: Early stopping is not available. Available metrics are: loss, root_mean_squared_error
  current = self.get_monitor_value(logs)
89520/89520 ————— 463s 5ms/step - loss: 98.0022 - root_mean_squared_error: 9.7845
Epoch 3/18
89520/89520 ————— 463s 5ms/step - loss: 96.7621 - root_mean_squared_error: 9.7140
Epoch 4/18
89520/89520 ————— 463s 5ms/step - loss: 96.2472 - root_mean_squared_error: 9.6744
Epoch 5/18
89520/89520 ————— 463s 5ms/step - loss: 92.9970 - root_mean_squared_error: 9.4994
Epoch 6/18
89520/89520 ————— 463s 5ms/step - loss: 92.8894 - root_mean_squared_error: 9.4917
Epoch 7/18
89520/89520 ————— 463s 5ms/step - loss: 93.2169 - root_mean_squared_error: 9.5105
Epoch 8/18
89520/89520 ————— 464s 5ms/step - loss: 93.4938 - root_mean_squared_error: 9.5135
Epoch 9/18
89520/89520 ————— 465s 5ms/step - loss: 92.1646 - root_mean_squared_error: 9.4400
Epoch 10/18
89520/89520 ————— 465s 5ms/step - loss: 91.2180 - root_mean_squared_error: 9.3865
Epoch 11/18
89520/89520 ————— 464s 5ms/step - loss: 90.3971 - root_mean_squared_error: 9.3418
Epoch 12/18
89520/89520 ————— 463s 5ms/step - loss: 89.9098 - root_mean_squared_error: 9.3116
Epoch 13/18
89520/89520 ————— 472s 5ms/step - loss: 89.0798 - root_mean_squared_error: 9.2615
Epoch 14/18
89520/89520 ————— 464s 5ms/step - loss: 88.5751 - root_mean_squared_error: 9.2281
Epoch 15/18
89520/89520 ————— 464s 5ms/step - loss: 88.5575 - root_mean_squared_error: 9.2250
Epoch 16/18
89520/89520 ————— 464s 5ms/step - loss: 87.9750 - root_mean_squared_error: 9.1906
Epoch 17/18
89520/89520 ————— 467s 5ms/step - loss: 89.2532 - root_mean_squared_error: 9.2559
Epoch 18/18
89520/89520 ————— 468s 5ms/step - loss: 88.8348 - root_mean_squared_error: 9.2323
```

train RMSE: 9.23

GRU

: 모델 검증

```
loss, rmse = final_gru_model.evaluate(x_test, t_test, verbose=1)
print('test loss(MSE) :', round(loss, 6))
print('test RMSE :', round(rmse, 2))
```

```
153463/153463 ————— 389s 3ms/step - loss: 103.6564 - root_mean_squared_error: 9.9351
test loss(MSE) : 121.74556
test RMSE : 10.87
```

test RMSE: 10.87

성능 비교 및 결론

	cell size	epoch	batch size	train RMSE	test RMSE
LSTM (1)	128	20	32	8.68	12.53
LSTM (2)	128	20	256	8.02	11.53
GRU	248	18	128	9.23	10.87 13.25% 개선

LSTM (1)

- 임의의 cell size, epoch, batch size 설정

LSTM (2)

- 결측치 처리 과정 개선
- 자동 하이퍼파라미터 튜닝 적용

GRU

- 결측치 처리 과정 개선
- 이상치 처리 과정 추가
- 자동 하이퍼파라미터 튜닝 적용